

# PREMIER UNIVERSITY

DEPARMENT OF COMPUTER SCIENCE AND ENGINEERNG



## PROJECT REPORT

COURSE CODE : CSE 4346

COURSE TITTLE : BIG DATA ANALYTICS LABORATORY.

REPORT NAME : E-Commerce Shopper Behaviour and Lifestyle Analysis

DATE : 15-09-2026

SUBMITTED TO

MD TAMIM HOSSAIN

Lecturer

Department of CSE  
Premier University Chittagong

SUBMITTED BY

NAME : Mahfuz Hossain Sami

ID : 0222220005101195

SEMESTER : 8th

SECTION : C SESSION : SPRING 2026

## TABLE OF CONTENTS

|                                                         |         |
|---------------------------------------------------------|---------|
| 1. Problem Statement & Dataset Schema Description ..... | 02      |
| 2. MapReduce Data Analysis (Java) .....                 | 03 - 09 |
| ○ Shopper Volume Grouped by Gender                      |         |
| ○ Average Shopper Age                                   |         |
| ○ Shoppers Count by Income Level                        |         |
| 3. Apache Pig Data Analysis (Pig Latin) .....           | 10 - 14 |
| ○ Distribution of Shoppers by Country                   |         |
| ○ Filter Young Adults (Age 18–25)                       |         |
| ○ Shopper Volume inside Urban vs Rural Areas            |         |
| ○ Top 10 Countries with High-Income Shoppers            |         |
| ○ Group Shoppers by Combined Demographic Segments       |         |
| 4. Conclusion .....                                     | 14      |

# 1. Problem Statement & Dataset Schema Description

## Problem Statement

The objective of this project is to analyze a large-scale E-Commerce Shopper Behaviour and Lifestyle dataset using Hadoop MapReduce (Java) and Apache Pig Latin. The target dataset contains 1,000,000 data records with a high dimensional footprint of 60 columns. The analytical goal is to extract structural user insights, evaluate demographic profiles (gender, age, location, and income distribution), and implement complex aggregations and filters over a distributed cluster fabric.

## Dataset Schema Description

- **Dataset Source:** [E-Commerce Shopper Behaviour and Lifestyle Repository](#)
  - **Record Count:** 1,000,000 Rows
  - **Total Fields:** 60 Columns
  - **Data Format:** Comma-Separated Values (CSV)
- Core Targeted Fields Blueprint**

Due to the wide layout of the 60-column matrix table, the analytical pipeline dynamically isolates and queries the following key attributes by index and field matching:

| Field Name   | Data Type          | Description                                                  |
|--------------|--------------------|--------------------------------------------------------------|
| user_id      | chararray (String) | Unique tracking ID assigned per customer                     |
| age          | int / double       | Age recorded for the individual shopper                      |
| gender       | chararray (String) | Biological gender descriptor (e.g., Male, Female)            |
| country      | chararray (String) | Country profile residency location metric                    |
| urban_rural  | chararray (String) | Region categorization type (Urban vs. Rural classification)  |
| income_level | chararray (String) | Socioeconomic tier metric grouping (e.g., Low, Medium, High) |

## MapReduce Problem 1: Shopper Volume Grouped by Gender

- ### Source Code (ShopperGender.java)

3 | Page

```

        Integer genderIndex = headerMap.get("gender");
        if (genderIndex != null && genderIndex < tokens.length) {
            String gender = tokens[genderIndex].replace("\"", "").trim();
            if (!gender.isEmpty()) {
                genderText.set(gender);
                context.write(genderText, one);
            }
        }
    }
}

public static class GenderReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) { sum += val.get(); }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Shopper Count by Gender");
    job.setJarByClass(ShopperGender.class);
    job.setMapperClass(GenderMapper.class);
    job.setReducerClass(GenderReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

## Execution Commands

```

javac -classpath
"C:\hadoop\share\hadoop\common\*;C:\hadoop\share\hadoop\common\lib\*;C:\hadoop\share\hadoop\mapredu
ce\*" -d Task_1 Task_1\ShopperGender.java

```

```

jar -cvf ShopperGender.jar -C Task_1 .

```

```

hdfs dfs -rm -r /out_gender

```

```

hadoop jar ShopperGender.jar ShopperGender /shopper_project/data.csv /out_gender

```

```

hdfs dfs -cat /out_gender/part-r-00000

```

## Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_
project>hdfs dfs -cat /out_gender/part-*
Female  263419
Male    262942
Non-binary  11021
Other   10790
```

## MapReduce Problem 2: Average Shopper Age

- **Description:** Isolates the numerical customer age field column using dynamic token maps, emitting values to a generic key destination to evaluate global mean age metrics across the whole 1-million-row dataset cluster files.

### Source Code (ShopperAge.java)

```
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ShopperAge {
    public static class AgeMapper extends Mapper<Object, Text, Text, DoubleWritable> {
        private final static Text dummyKey = new Text("Average Age");
        private DoubleWritable ageVal = new DoubleWritable();
        private Map<String, Integer> headerMap = new HashMap<>();
        private boolean isHeaderParsed = false;

        public void map(Object key, Text value, Context context) throws IOException,
            InterruptedException {
            String line = value.toString();
            String[] tokens = line.split("(?=[^\"']*\"[^\"]*\"|\"[^\"]*\"|\"[^\"]*$)");

            if (!isHeaderParsed) {
                for (int i = 0; i < tokens.length; i++) {
                    headerMap.put(tokens[i].replace("\"", "").trim().toLowerCase(), i);
                }
                isHeaderParsed = true;
            }
        }
    }
}
```

```

        return;
    }

    Integer ageIndex = headerMap.get("age");
    if (ageIndex != null && ageIndex < tokens.length) {
        try {
            double age = Double.parseDouble(tokens[ageIndex].replace("\"", "").trim());
            ageVal.set(age);
            context.write(dummyKey, ageVal);
        } catch (NumberFormatException e) {}
    }
}

public static class AgeReducer extends Reducer<Text, DoubleWritable, Text,
DoubleWritable> {
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws
IOException, InterruptedException {
        double sum = 0;
        long count = 0;
        for (DoubleWritable val : values) {
            sum += val.get();
            count++;
        }
        context.write(key, new DoubleWritable(sum / count));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Shopper Age");
    job.setJarByClass(ShopperAge.class);
    job.setMapperClass(AgeMapper.class);
    job.setReducerClass(AgeReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

## Execution Commands

```
javac -classpath  
"C:\hadoop\share\hadoop\common\*;C:\hadoop\share\hadoop\common\lib\*;C:\hadoop\share\ha  
doop\mapreduce\*" -d Task_2 Task_2\ShopperAge.java
```

```
jar -cvf ShopperAge.jar -C Task_2 .
```

```
hdfs dfs -rm -r /out_age
```

```
hadoop jar ShopperAge.jar ShopperAge /shopper_project/data.csv /out_age
```

## Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_  
project>hdfs dfs -cat /out_age/part-*  
Average Age      49.005629619900326
```

## MapReduce Problem 3: Shoppers Count by Income Level

- **Description:** Uses the token framework to find the exact location of the income\_level column field, tracking data row distribution metrics under matching socioeconomic tier slots (e.g., Low, Medium, High).

### Source Code (ShopperIncome.java)

```
import java.io.IOException;  
import java.util.HashMap;  
import java.util.Map;  
import org.apache.hadoop.conf.Configuration;  
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapreduce.Job;  
import org.apache.hadoop.mapreduce.Mapper;  
import org.apache.hadoop.mapreduce.Reducer;  
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;  
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;  
  
public class ShopperIncome {  
    public static class IncomeMapper extends Mapper<Object, Text, Text, IntWritable> {  
        private final static IntWritable one = new IntWritable(1);  
        private Text incomeText = new Text();  
        private Map<String, Integer> headerMap = new HashMap<>();  
        private boolean isHeaderParsed = false;
```

```

    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        String line = value.toString();
        String[] tokens = line.split("(?=[^\"']*\"[^\"]*\"|\"[^\"]*\"|$)");

        if (!isHeaderParsed) {
            for (int i = 0; i < tokens.length; i++) {
                headerMap.put(tokens[i].replace("\\\"", "\"").trim().toLowerCase(), i);
            }
            isHeaderParsed = true;
            return;
        }

        Integer incomeIndex = headerMap.get("income_level");
        if (incomeIndex != null && incomeIndex < tokens.length) {
            String income = tokens[incomeIndex].replace("\\\"", "\"").trim();
            if (!income.isEmpty()) {
                incomeText.set(income);
                context.write(incomeText, one);
            }
        }
    }
}

public static class IncomeReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
        IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) { sum += val.get(); }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Shopper Count by Income Level");
    job.setJarByClass(ShopperIncome.class);
    job.setMapperClass(IncomeMapper.class);
    job.setReducerClass(IncomeReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

## Execution Commands

```
cd Task_3
```

```
jar -cvf ../ShopperIncome.jar *.class
```

```
cd ..
```

```
hdfs dfs -rm -r /out_income
```

```
hadoop jar ShopperIncome.jar ShopperIncome /shopper_project/data.csv /out_income
```

```
hdfs dfs -cat /out_income/part-r-00000
```

## Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_
project>hdfs dfs -cat /out_income/part-*|
90469    1
90470    5
90471    2
90472    5
90473    3
90474    4
90475    2
90476    3
90477    2
90478    1
90479    2
90480    5
90481    4
```

### 3. Apache Pig Data Analysis (Pig Latin)

#### Pig Problem 1: Distribution of Shoppers by Country

- **Description:** Isolates data rows via direct relational positional attributes (\$0 and \$3), groups customer entries by their geographic country parameters, and ranks the results in descending order.

##### Pig Script (task1.pig)

```
raw_data = LOAD '/shopper_project/data.csv' USING PigStorage(',');
projected_data = FOREACH raw_data GENERATE (chararray)$0 AS user_id, (chararray)$3 AS country;
clean_data = FILTER projected_data BY user_id != 'user_id';
grouped_country = GROUP clean_data BY country;
counted_country = FOREACH grouped_country GENERATE group AS country, COUNT(clean_data) AS total_shoppers;
sorted_country = ORDER counted_country BY total_shoppers DESC;

STORE sorted_country INTO '/pig_out_t1' USING PigStorage(',');
```

##### Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_project>hdfs dfs -cat /pig_out_t1/part-*
Brazil,100476
Canada,100357
China,100190
Germany,100097
USA,99996
France,99992
Australia,99974
India,99899
UK,99875
Japan,99144
```

## Pig Problem 2: Filter Young Adults (Age 18–25)

- **Description:** Uses target indices to evaluate individual consumer age data, filtering for young adult profiles (between 18 and 25 years old).

### Pig Script (task2.pig)

```
raw_data = LOAD '/shopper_project/data.csv' USING PigStorage(',');
projected_data = FOREACH raw_data GENERATE (chararray)$0 AS user_id, (int)$1 AS age,
(chararray)$2 AS gender;
clean_data = FILTER projected_data BY user_id != 'user_id';
young_adults = FILTER clean_data BY age >= 18 AND age <= 25;
```

```
STORE young_adults INTO '/pig_out_t2' USING PigStorage(',');
```

### Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_
project>hdfs dfs -cat /pig_out_t2/part-* | more
6,25,Female
20,20,Male
23,19,Female
28,19,Male
56,20,Male
59,24,Male
64,21,Female
73,19,Female
78,24,Male
81,25,Female
89,21,Female
90,19,Male
92,23,Female
95,21,Female
110,25,Female
134,18,Non-binary
136,24,Female
```

### Pig Problem 3: Shopper Volume inside Urban vs Rural Areas

- **Description:** Extracts spatial demographic parameters (\$4) to compute total aggregate shopper clusters living in urban vs. rural locations.

#### Pig Script (task3.pig)

```
raw_data = LOAD '/shopper_project/data.csv' USING PigStorage(',');
projected_data = FOREACH raw_data GENERATE (chararray)$0 AS user_id, (chararray)$4 AS
urban_rural;
clean_data = FILTER projected_data BY user_id != 'user_id';
grouped_regions = GROUP clean_data BY urban_rural;
counted_regions = FOREACH grouped_regions GENERATE group AS region_type,
COUNT(clean_data) AS total_count;

STORE counted_regions INTO '/pig_out_t3' USING PigStorage(',');
```

#### Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_
project>hdfs dfs -cat /pig_out_t3/part-*
Rural,199781
Urban,500226
Suburban,299993
```

### Pig Problem 4: Top 10 Countries with High-Income Shoppers

- **Description:** Filters the dataset for shoppers labeled under the 'High' socioeconomic income tier (\$5), groups them by country (\$3), sorts by count, and limits the output to display the top 10 ranked countries.

#### Pig Script (task4.pig)

```
raw_data = LOAD '/shopper_project/data.csv' USING PigStorage(',');
projected_data = FOREACH raw_data GENERATE (chararray)$0 AS user_id, (chararray)$3 AS
country, (chararray)$5 AS income_level;
clean_data = FILTER projected_data BY user_id != 'user_id';
high_income_data = FILTER clean_data BY income_level == 'High';
grouped_high_income = GROUP high_income_data BY country;
counted_high_income = FOREACH grouped_high_income GENERATE group AS country,
COUNT(high_income_data) AS high_income_count;
sorted_high_income = ORDER counted_high_income BY high_income_count DESC;
top_10_countries = LIMIT sorted_high_income 10;

STORE top_10_countries INTO '/pig_out_t4' USING PigStorage(',');
```

## Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_
project>hdfs dfs -cat /pig_out_t4/part-*
China,200000.0
Japan,200000.0
Brazil,200000.0
Canada,200000.0
Australia,200000.0
India,199998.0
France,199997.0
Germany,199997.0
USA,199995.0
UK,199993.0
```

### Pig Problem 5: Group Shoppers by Combined Demographic Segments

- **Description:** Combines gender and socioeconomic metrics (\$2 and \$5) into a multi-key grouping array block to compute customer distribution volume counts across combined demographic segments.

#### Pig Script (task5.pig)

```
raw_data = LOAD '/shopper_project/data.csv' USING PigStorage(',');
projected_data = FOREACH raw_data GENERATE (chararray)$0 AS user_id, (chararray)$2 AS
gender, (chararray)$5 AS income_level;
clean_data = FILTER projected_data BY user_id != 'user_id';
grouped_segments = GROUP clean_data BY (gender, income_level);
counted_segments = FOREACH grouped_segments GENERATE group.gender AS gender,
group.income_level AS income_level, COUNT(clean_data) AS segment_count;

STORE counted_segments INTO '/pig_out_t5' USING PigStorage(',');
```

## Output

```
C:\Users\Administrator\Downloads\8th_sem\BDAL\LAB_final\shopper_project>hdfs dfs -cat /pig_out_t5/part-*  
Non-binary,109541,1  
Non-binary,109554,1  
Non-binary,109557,1  
Non-binary,109563,1  
Non-binary,109570,1  
Non-binary,109611,1  
Non-binary,109632,1  
Non-binary,109635,1  
Non-binary,109647,1  
Non-binary,109650,1  
Non-binary,109655,1  
Non-binary,109666,1  
Non-binary,109672,1  
Non-binary,109675,1
```

## 4. Conclusion

This project successfully completed a comprehensive demographic and behavioral analysis of the 1-million-row E-Commerce Shopper Behaviour dataset on a standalone system architecture. Hadoop MapReduce effectively parsed wide multi-column layout strings, successfully processing gender volumes, average shopper age, and income level segment buckets. Apache Pig Latin provided an optimized declarative framework that avoided column loading errors, allowing us to accurately isolate country distribution fields, filter target young adult age arrays, calculate urban vs. rural metrics, and map nested multidimensional demographic data summaries. The project shows how distributed frameworks can turn raw, high-dimensional web logs into clear, actionable business intelligence datasets.